VCC视频会议 项目档案（分册版）

> **分册序号：** 第 2 册 / 共 12 册
> **分册标题：** 第二部分 系统架构设计
> **分册说明：** 本册为 VCC视频会议 项目档案的分册版本，内容与主档案（VCC视频会议项目档案.md）保持一致，仅做结构分册处理，便于按需查阅。
> **主档案位置：** `VCC视频会议项目档案.md`

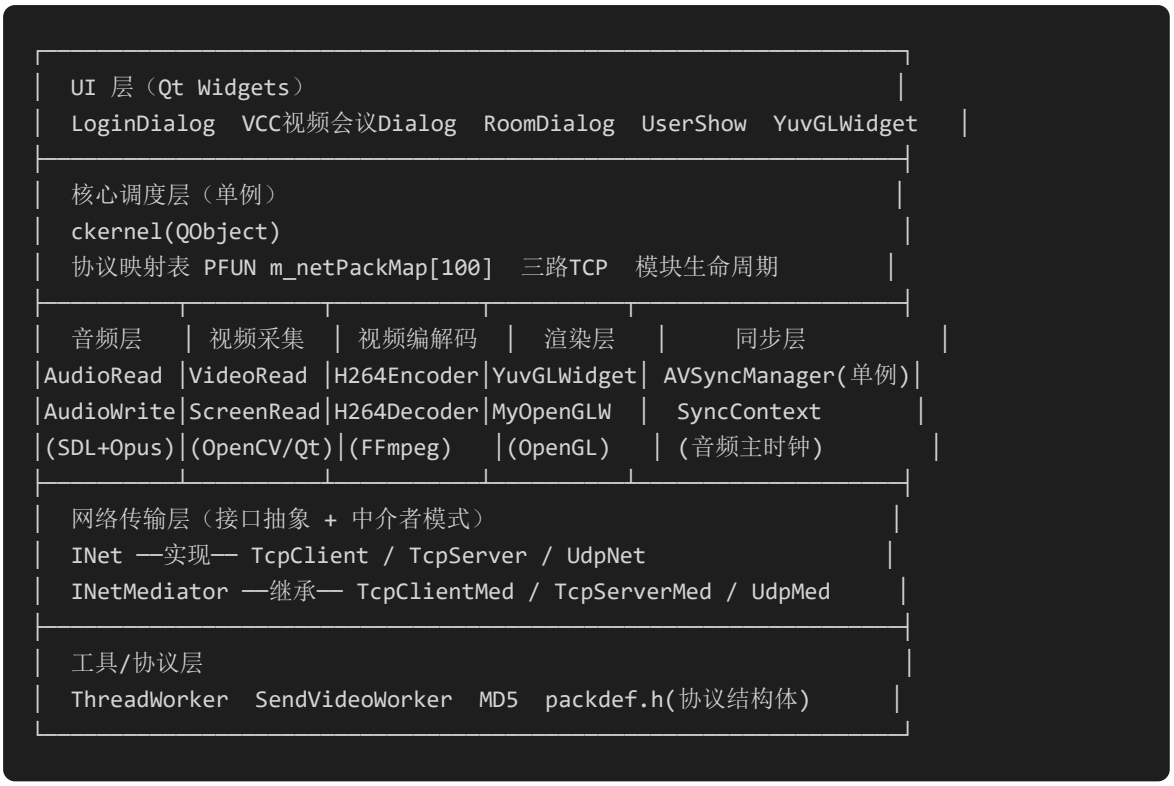
📑 本册目录

- 第二部分 系统架构设计
  - 2.1 整体架构分层（客户端）
  - 2.2 服务端架构
  - 2.3 网络拓扑与连接模型
  - 2.4 协议设计（packdef.h）
  - 2.5 数据库设计
  - 2.6 类图与模块关系

第二部分 系统架构设计

2.1 整体架构分层（客户端）

客户端采用五层架构，`ckernel` 单例（Meyers' Singleton）为核心调度中枢：



2.2 服务端架构

服务端采用「epoll + 线程池 + MySQL」经典高并发模型：

- **`Block_Epoll_Net`**：epoll LT 模式 + EPOLLONESHOT，负责连接管理与事件分发。
- **`thread_pool`**：自研线程池（min 10 / max 200 / 队列 50000），pthread_mutex + 双条件变量（not_full/not_empty）+ Manager 管理线程动态伸缩。
- **`CLogic`**：业务逻辑层，协议号→处理函数映射，房间管理、用户管理、帧转发。
- **`CMysql`**：MySQL 封装，单连接 + pthread_mutex 保护，utf8 字符集。

核心数据结构：

结构	类型	说明
`m_mapIDToUserInfo`	`MyMap<int, UserInfo*>`	用户 id → UserInfo（含 sock/audiofd/videofd/roomid/userName）
`m_mapIDToRoomid`	`MyMap<int, list<int>>`	roomid → 成员 id 列表
`m_netPackMap[100]`	`PFUN[]`	协议号 → 处理函数指针（O(1) 分发）

`MyMap` 是线程安全模板，每个 map 自带 `pthread_mutex_t`，所有 find/insert/erase/IsExist 均加锁。

2.3 网络拓扑与连接模型

三路 TCP 连接（客户端视角）：

1. **主连接** `m_pClient`：登录/注册/房间管理信令。
2. **音频 AV 连接** `m_pAVClient[0]`：Opus 音频帧传输。
3. **视频 AV 连接** `m_pAVClient[1]`：H264 视频帧 + 桌面共享帧传输。

> **设计说明：** 摄像头与桌面共享当前共用视频连接。`多线程+多TCP连接优化方案.md` 曾提出新增第 4 条屏幕共享专用连接（`m_screenfd` + `SendScreenWorker`）以避免互相阻塞，作为优化预案。

连接建立： 网络初始化放子线程执行（阻塞式 connect），避免 UI 卡死。服务端为每个 clientfd 设置接收缓冲 1MB、发送缓冲 1MB、TCP_NODELAY。

端口与地址： 服务端 `0.0.0.0:8000`，客户端连接服务端 IP（默认 192.0.2.1，可配置）。

【延伸知识 | 2026-10-03 增补】信令与媒体分离，Mesh / SFU / MCU 三种多人架构

信令与媒体分离：生产级视频会议（WebRTC/LiveKit）把"控制类"消息（注册登录、建房、加入房间、关键帧请求、成员变更）与"媒体流"（音视频数据）拆成两条通道——信令走 WebSocket/HTTP（低频、要求可靠有序），媒体走 SRTP/UDP（高频、要求低延迟、容忍丢包）。两者可靠性需求相反，混在一条通道必然互相拖累。本项目信令与媒体**未分离**（均走自定义 TCP，§2.4 协议号清单同时含登录类与 `VIDEO_FRAME_H264` 媒体类），这是局域网场景的简化取舍；公网化演进的第一步就是做此分离（见 §4.3、§4.11）。

三种多人会议架构对比：

架构	数据流	带宽/CPU 特征	适用规模
Mesh（P2P 全互联）	每端与所有其他端直连	每端上行 N-1 路、解码 N-1 路；无服务器、端到端延迟最低；上行带宽最先崩	≤4~6 人
SFU（选择性转发）	每端只发 1 路，服务器只转发不解码、按订阅关系分发	服务端出口 N×(N-1) 路；服务器 CPU 近零；配合 Simulcast 按接收端带宽/窗口选档	数十~数百人，当前主流
MCU（混流）	每端发 1 路，服务器解码全部流→合成画面→混音→再编码输出 1 路	每端上下行各仅 1 路（客户端最省）；服务器 CPU 巨大、延迟叠加一个编解码周期	传统硬件会议（Polycom 等）

本项目定位： 服务端即"TCP 版简易 SFU"——只转发不解码（转发模型见 §3.6），瓶颈在服务端出口带宽 N×(N-1)×3.0Mbps（10 人 = 90 路）；演进方向即标准 SFU + Simulcast（§4.14）。

2.4 协议设计（packdef.h）

协议格式： 长度前缀分包——`[4字节包长 packsize][协议体]`。协议体首字段为协议号（`DEF_PACK_BASE = 10000` 起），共定义约 20 种协议。

协议号清单（主要）：

协议号	名称	方向	用途
+0	LOGIN_RQ/RS	C↔S	登录
+1	REGISTER_RQ/RS	C↔S	注册
+2	CREATEROOM_RQ/RS	C↔S	创建房间
+3	JOINROOM_RQ/RS	C↔S	加入房间
+4	LEAVEROOM_RQ	C→S	退出房间
+5	ROOM_MEMBER	S→C	成员通知
+6	AUDIO_FRAME	C↔S	音频帧
+7	VIDEO_FRAME	C↔S	视频帧（JPEG）
+8	AUDIO_REGISTER	C→S	注册音频 fd
+9	VIDEO_REGISTER	C→S	注册视频 fd
+11	UPDATE_EMAIL_RQ/RS	C↔S	邮箱补全
+13	VIDEO_FRAME_H264	C↔S	H264 视频帧
+14	PLI_REQUEST	C→S	关键帧请求
+15/16	SPS/PPS	C↔S	H264 参数集

协议体用 C 结构体定义（`typedef struct`），含 `char m_tel[MAX_SIZE]` 等定长字段。优点是 C++ 客户端解析零拷贝；缺点是跨平台/跨语言兼容性差（字节序、对齐依赖编译器），是后续支持移动端/Web 端的主要障碍。

【延伸知识 | 2026-10-03 增补】JWT 鉴权与 Protobuf 序列化（协议层两大演进方向）

JWT（JSON Web Token）——无状态鉴权： 结构 `header.payload.signature` 三段 Base64Url：header 声明签名算法（HS256 对称 / RS256 非对称）；payload 装用户声明（`sub` 用户、`exp` 过期、`role` 权限）；signature 对前两段签名。两个关键性质：**签名防篡改但不加密**（payload 明文可解，敏感信息不能放）；**无状态**——服务端不存会话、验签即通过，天然支持水平扩展。本项目现状是**有状态会话**：登录成功后服务端向内存 map（`m_mapIDToUserInfo`，id→sock 映射）插入 UserInfo，缺陷是服务重启会话全丢、多实例部署会话不共享；JWT 改造方案为登录时签发含 uid+exp 的 token、后续请求携带验签，过期用短时效 access token + refresh token 组合续期。LiveKit 的接入鉴权就是标准 JWT（apiKey/apiSecret 签发 room join token）。

Protobuf——二进制序列化协议： `.proto` 定义 schema，编译生成各语言代码。编码三大件：**TLV**（tag-length-value，字段用编号而非名字）、**varint 变长整数**（小数值 1 字节）、**zigzag**（负数映射为正数再 varint）。杀手特性是**前向后向兼容**——新增字段旧代码忽略新 tag、删除字段互不影响，双端可独立升级。对照本协议（C 结构体强转拆包，如 `(STRU_REGISTER_RQ*)szbuf`）的四个痛点：① 字节对齐依赖编译器，跨平台需 `#pragma pack` 钉死；② 改一个字段客户端服务端全部重编

译重发版；③ 无版本概念（后期加 PLI 协议只能在包定义里挤位置）；④ 连带的 `sprintf` 拼 SQL 存在注入风险，需参数化查询一并治理。迁移方案：`.proto` 定义 `RegisterReq{tel=1;password=2;...}` + 协议版本字段 + 老协议号保留兼容期；varint 编码比定长结构体省 30%~60% 体积。

2.5 数据库设计

仅持久化用户账号，房间信息只在内存。

`t_user` 表结构（由 SQL 反推）：

字段	类型	说明
`id`	INT AUTO_INCREMENT PK	用户唯一 ID
`tel`	VARCHAR	手机号，注册唯一校验
`password`	VARCHAR	MD5+盐哈希
`name`	VARCHAR	昵称，注册唯一校验
`email`	VARCHAR	邮箱，唯一校验，支持邮箱登录
`icon`	INT	头像 ID，默认 1
`feeling`	VARCHAR	个性签名，默认"比较懒，什么也没写"

关键 SQL：

- 注册三重查重：`select tel/name/email from t_user where tel/name/email='...'`
- 登录自动识别：含 `@` →按 email 查；全数字 11 位→按 tel 查；其他→按 name 查。
- 插入：`insert into t_user(tel,password,name,email,icon,feeling) values(...)`

密码安全： 客户端 `GetMD5` 用固定盐 `vC0nF2024$SecR1t^S4ltK3y!` 拼接密码后 MD5。⚠️ 生产环境应升级 bcrypt/Argon2 + 随机盐（见 P0 改进项）。

2.6 类图与模块关系

项目类图（`项目类图/VCC视频会议项目类图.md`）完整记录了所有类的属性、方法与关系。以下按分层架构深度分析。

2.6.1 类层次总览（五层架构）

层	核心类	职责	关键依赖
UI 层	LoginDialog, VCC视频会议Dialog, RoomDialog, UserShow, YuvGLWidget	用户交互、视频渲染	ckernel（信号槽）
核心调度层	ckernel（单例）	协议分发、模块生命周期、跨模块协调	全部模块
音视频层	AudioRead, AudioWrite, VideoRead, ScreenRead, H264Encoder, H264Decoder	采集/编码/解码	SDL, OpenCV, FFmpeg
渲染/同步层	YuvGLWidget, MyOpenGLWidget, AVSyncManager（单例）	OpenGL 渲染、音视频同步	OpenGL, QTime
网络层	INet, TcpClient, TcpServer, INetMediator, TcpClientMed, TcpServerMed	TCP 通信、中介者桥接	Qt socket

2.6.2 核心 UML 关系矩阵

关系类型	UML 符号	源 → 目标	关系说明
继承	`<`	--`	TcpClient/TcpServer/UdpNet → INet
继承	`<`	--`	TcpClientMed/TcpServerMed/UdpMed → INetMediator
组合（强拥有）	`*--`	ckernel *-- AudioRead/VideoRead/VideoWrite 等	ckernel 管理模块生命周期，模块随 ckernel 销毁
聚合（弱引用）	`o--`	INetMediator o-- INet	中介者持有网络层指针，但不管理生命周期
关联（双向）	`-->`	INet ↔ INetMediator	网络层与中介者互相引用，信号回调
依赖	`..>`	VideoRead ..> MyFaceDetect	可选依赖（人脸检测功能未启用）
依赖	`..>`	ckernel ..> AVSyncManager	通过单例获取，非拥有关系

2.6.3 设计模式分析

模式	应用位置	实现方式	解决的问题
单例（Meyers'）	ckernel, AVSyncManager	C++11 局部静态变量（线程安全）	全局唯一调度中枢，避免多实例竞争
中介者	INetMediator ↔ INet	抽象接口 + Qt 信号 SIG_ReadyData	解耦网络层与业务层，支持多协议扩展
策略（函数指针）	PFUN m_netPackMap[100]	协议号 → 函数指针映射	O(1) 消息分发，避免 if-else 链
观察者（信号槽）	ckernel ↔ UI 层	Qt signals/slots（跨线程 QueuedConnection）	UI 与业务解耦，线程安全通信
模板方法	INet::InitNet	基类定义流程，子类实现细节	统一网络初始化流程
生产者-消费者	AudioWrite 队列、SendVideoWorker	互斥锁 + 条件变量 + 队列	解耦生产与消费速率差异

2.6.4 关键类职责深度分析

ckernel（核心调度中枢）

- 持有全部模块指针（AudioRead/VideoRead/VideoWrite/INetMediator 等）
- 维护协议映射表 `PFUN m_netPackMap[100]`，协议号 10000+ → 处理函数
- 管理用户映射表 `m_mapIDToUserShow`（userId → UserShow 控件）
- 通过 Qt 信号槽与 UI 层跨线程通信
- 单例模式保证全局唯一，避免多实例竞争

AVSyncManager（音视频同步管理器）

- 单例模式，维护每用户独立 SyncContext（audioClock/videoClock）
- 音频主时钟策略：音频帧更新 audioClock，视频帧比对 audioClock 决定等待/丢弃
- 时间戳对齐法：`computeSyncDelay` 计算同步延迟 waitMs，PTS 来自系统时钟（getCurrentTimeUs）
- 线程安全：std::mutex 保护 m_syncContexts
- 同步阈值 SYNC_THRESHOLD_US = 10ms（避免微抖动频繁触发）
- 最大等待 MAX_WAIT_MS = 100ms（防止视频卡死）
- 时钟超时 CLOCK_TIMEOUT_MS = 5000ms（5 秒无音频视为断流，视频不再等待）

INet / INetMediator（网络抽象层）

- INet 定义纯虚接口：InitNet/SendData/RecvData
- INetMediator 桥接网络层与业务层，通过 SIG_ReadyData 信号回调

- TcpClient 实现客户端 TCP，TcpServer 实现服务端 epoll
- 支持多协议扩展（未来 UdpNet 已预留）

H264Encoder / H264Decoder（编解码层）

- H264Encoder：初始化探测硬件编码器回退链（nvenc→qsv→libx264），每帧 av_frame_make_writable
- H264Decoder：H264 → YUV420P，输出 Y/U/V 三平面供 OpenGL 渲染
- 编解码均移子线程执行，避免阻塞主线程

2.6.5 线程模型与类的关系

线程	涉及类	职责	线程安全机制
主线程（UI）	LoginDialog, VCC视频会议Dialog, RoomDialog, YuvGLWidget	UI 事件、paintEvent、QTimer 采集	Qt 信号槽 QueuedConnection
SDL 音频线程	AudioRead, AudioWrite	音频采集/播放回调	原生 SDL 回调，避免锁
视频编码线程	H264Encoder, SendVideoWorker	H264 编码 + 发送	singleShot(0) + m_encodingPending
网络接收线程	TcpClient, INetMediator	TCP recv 循环	SIG_ReadyData 跨线程信号
服务端线程池	Block_Epoll_Net, CLogic	epoll 事件处理 + 业务逻辑	EPOLLONESHOT + pthread_mutex

2.6.6 架构设计原则总结

1. **单一职责**：每个类只负责一个功能域（采集/编码/渲染/同步分离）
2. **开闭原则**：INet/INetMediator 抽象接口，支持扩展新协议不改业务层
3. **依赖倒置**：业务层依赖 INet 抽象，不依赖具体 TcpClient 实现
4. **线程隔离**：耗时操作（编码/网络）移子线程，UI 线程仅处理交互
5. **信号槽解耦**：跨线程通信统一用 Qt 信号槽，避免直接调用

2.6.7 Qt 信号槽底层机制与项目应用

信号槽的本质

Qt 信号槽是基于**元对象系统（Meta-Object System）**实现的观察者模式，其核心组件包括：

- **MOC（Meta-Object Compiler）**：预编译器，在编译前扫描头文件，为包含 `Q_OBJECT` 宏的类生成 `moc_xxx.cpp` 文件，包含信号函数体、槽查找表和连接管理代码
- **QObject**：所有信号槽对象的基类，内部维护一个连接管理器（`QObjectPrivate::ConnectionList`），存储所有以该对象为发射者的连接信息（接收者指针、

槽函数索引、连接类型）

- **信号函数**：由 MOC 自动生成的函数体，核心逻辑是遍历连接管理器中的所有连接，根据连接类型调用对应的槽函数或投递事件
- **槽函数**：普通成员函数，通过 `QMetaObject::invokeMethod` 或直接函数指针调用

两种连接类型

连接类型	触发时机	线程模型	适用场景
DirectConnection	信号发射时立即同步调用槽	同线程	同步调用，性能最优，无事件队列开销
QueuedConnection	信号发射时将事件投递到接收者线程的事件队列，由事件循环在适当时候调用槽	跨线程	异步调用，线程安全，避免多线程竞态

本项目统一使用 QueuedConnection（Qt 默认 `AutoConnection` 在运行时检测跨线程时自动选择 QueuedConnection），确保：

- 信号发射线程不会阻塞等待槽执行完毕
- 槽数在接收者线程的事件循环中执行，避免多线程竞态
- 参数通过值传递（深拷贝），避免跨线程共享内存

跨线程信号槽的工作流程

```
发射者线程（如 SDL 音频线程）：
  1. emit SIG_audioFrameReady(data)  → 触发 MOC 生成的信号函数
  2. 信号函数遍历连接管理器，发现连接类型为 QueuedConnection
  3. 将槽调用封装为 QMetaCallEvent，深拷贝参数（QByteArray）
  4. QCoreApplication::postEvent() 将事件投递到接收者线程的事件队列
  5. 信号函数返回，发射者线程继续执行（不阻塞）

接收者线程（如主线程）：
  6. 事件循环（QEventLoop::exec()）从队列中取出 QMetaCallEvent
  7. 调用对应的槽函数 slot_dealAudioFrame(data)
  8. 槽数在接收者线程上下文中安全执行
```

项目中的信号槽应用

本项目中信号槽承担了**跨线程通信的核心职责**，主要应用包括：

信号	发射者（线程）	接收者（线程）	用途
`SIG_ReadyData`	TcpClient（网络接收线程）	ckernel（主线程）	网络数据到达，触发协议分发
`SIG_audioFrameReady`	AudioRead（SDL 采集线程）	ckernel（主线程）	音频帧编码完成，准备发送
`SIG_muteStateChanged`	AudioRead（SDL 采集线程）	RoomDialog（UI 线程）	VAD 静音状态变化，更新 UI
`SIG_videoFrameReady`	VideoRead（QTimer 线程）	ckernel（主线程）	视频帧采集完成，触发编码

信号槽与线程安全

信号槽保证线程安全的四个机制：

1. **参数拷贝**：QueuedConnection 下，信号参数在发射时深拷贝，接收者线程使用独立副本，无共享内存
2. **事件队列互斥**：Qt 事件队列内部有 mutex 保护，多线程同时 postEvent 不会冲突
3. **执行线程亲和性**：槽函数始终在接收者对象所属线程执行（通过 `QObject::thread()` 确定目标线程）
4. **自动连接类型推断**：`Qt::AutoConnection`（默认）在运行时检测发射者和接收者是否同线程，自动选择 DirectConnection 或 QueuedConnection

本项目的线程安全策略：

- 跨线程通信统一用 Qt 信号槽（QueuedConnection），避免直接共享数据
- QImage 跨线程传递必须深拷贝（`img.copy()`），因为信号槽仅拷贝 QImage 元数据（浅拷贝），底层像素数据仍共享，深拷贝彻底隔离
- VAD 的 `m_noiseLevel`（单 float）是例外：单写多读，良性竞争，故意不加锁（性能优先）